

벡터DB / GraphDB 구축 결과서

SKN Final Project 최종 산출물 — CopyLane

팀장 **오한빈**
팀원 **박수진 · 권소라 · 소성민 · 이서은**
작성자 **오한빈** (팀장)
도메인 제안 **권소라**
개정 **2026-10-21** · 기준 커밋 **ksr b54246d**

목차

1. 한 장 요약 4

2. 왜 별도 벡터DB · GraphDB 를 두지 않았나 5

3. 저장 구조 6

4. 적재 결과 8

4-1. 청크 (이 PC DB · 2026-10-21 조회 · 정본 수는 원장 10-13(B) ①) 8

4-2. 함께 적재되는 관계형 표 (판정이 벡터와 같이 읽는 것) 8

5. 검색 구조 10

6. 측정 결과 12

6-1. 검색 품질 — 탐침 12

6-2. 응답시간 12

7. 지키는 장치 14

8. 남은 것 15

작성 오한빈(팀장) · **개정** 2026-10-21 · **기준 커밋** `ksr b54246d` (main `2f1e86d` 포함)

앞 판 — 2026-10-06 제출본(청크 6,667 기준). 이 판은 10-13 청크 재생성(6,686)과 10-19 참고 조문 재배치, 10-20 응답 시간 측정까지 반영했다.

GraphDB 는 쓰지 않는다. 이 프로젝트에는 그래프 데이터베이스가 없다. 「그래프」는 판정 흐름(LangGraph)을 가리키고, 저장소는 관계형 + 벡터 한 벌이다(§2). 조문 사이의 관계(법 → 조 → 항 → 호 → 목 · 별표)는 관계형 칸과 인용 좌표로 든다.

이 문서가 다루는 것 — ① 벡터 저장 구조와 그렇게 정한 까닭 ② 무엇을 얼마나 넣었나 ③ 검색이 어떻게 도나 ④ 측정 결과 ⑤ 남은 것.

수치의 출처. 스키마의 정본은 `db/schema.sql`, 수치의 정본은 `docs/00_사실원장.md` (이하 「원장」)다. 절마다 날짜와 기기를 적는다. 「이 PC DB」는 2026-10-21 에 개정 작업 PC 의 DB(복제본)를 읽기 전용 SELECT 로 조회한 값이고, 정본 기기(클론 B)의 원장 수와 맞는지 함께 적었다.

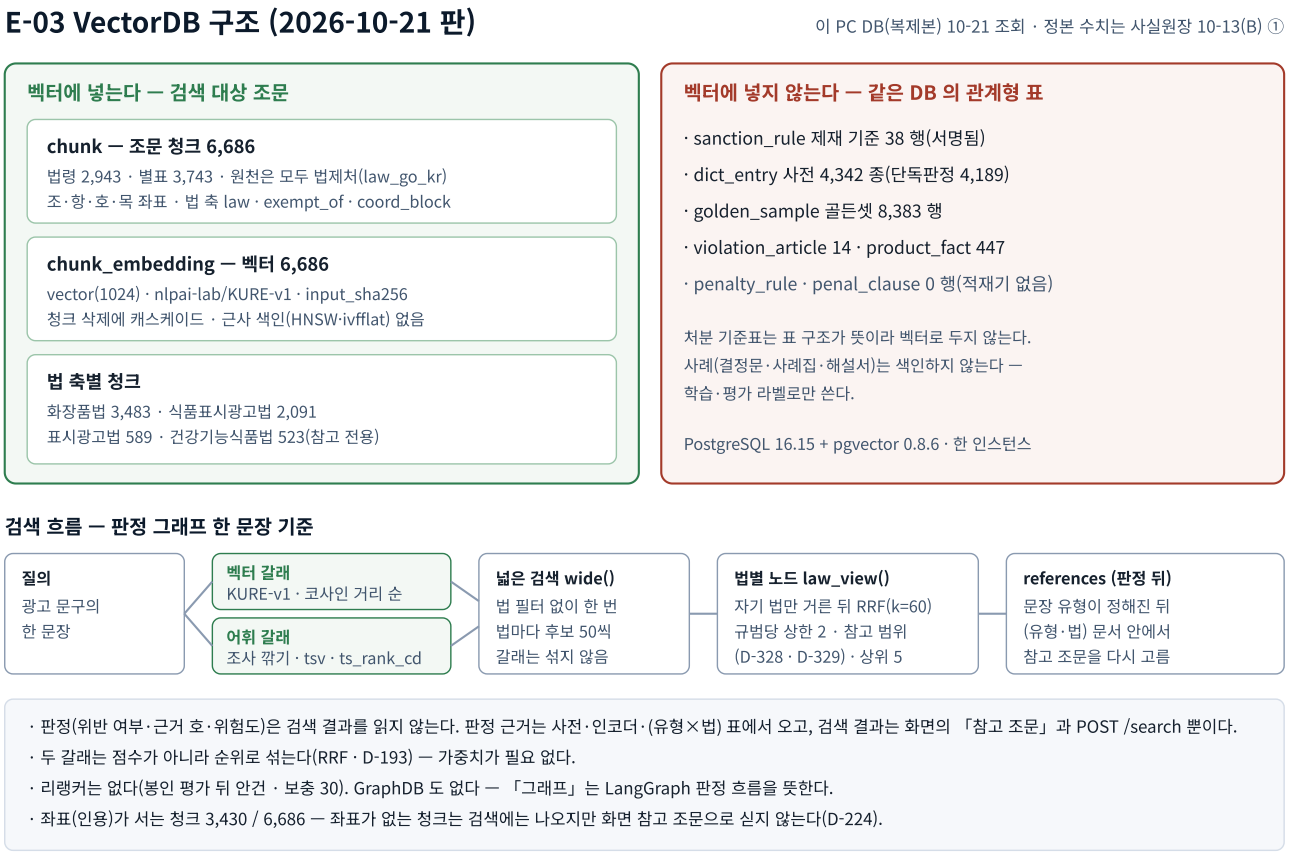
1. 한 장 요약

물음	답
벡터 저장소	**pgvector 0.8.6** — 관계형 DB 와 같은 **PostgreSQL 16.15** 인스턴스(D-41)
임베딩 모델	`nlpai-lab/KURE-v1` · 1024 차원 · 코사인 거리 · 파인튜닝 안 함
무엇을 넣었나	법령 · 행정규칙 · 별표의 **조문 청크 6,686**(임베딩 6,686)
검색	벡터 + 어휘 **두 갈래 → 순위 융합(RRF · D-193)** · 판정 그래프는 법 필터 없이 한 번 넓게 찾고 법별 노드가 자기 법 것만 거른다(D-291)
검색 결과의 쓰임	화면의 **참고 조문**과 `POST /search` 뿐이다. 판정(위반 여부 · 근거 호 · 위험도)은 검색 결과를 읽지 않는다
참고 조문 고르기	2026-10-19 부터 **판정 뒤에** 문장 유형이 정해진 다음 (유형 · 법) 문서 안에서 고른다(노드 `references` · 보충 30)
벡터 인덱스	**만들지 않았다 — 판단이다** · 이 규모에서는 전수 비교(순차 스캔)
리랭커	없다 — 봉인 평가 뒤 안건(보충 30)
응답 시간	판정 요청 1번 **542 ms** · 그중 검색 노드 291 ms(원장 10-20 ① · 정본 기기)
GraphDB	없다

2. 왜 별도 벡터DB · GraphDB 를 두지 않았나

- **삭제가 따라가야 한다** — 원천을 지우면 그 원천에서 나온 청크와 벡터가 함께 사라져야 한다(데이터 거버넌스의 요구). 같은 인스턴스에 두면 외래키 캐스케이드가 그대로 걸린다. 벡터DB 를 따로 두면 두 저장소의 삭제를 맞추는 코드가 생긴다(D-41).
- **검색 질의가 거버넌스 조건을 든다** — 모든 검색 질의가 「이 원천을 RAG 용도(`U2_rag`)로 써도 되는가」의 조인(`source_use.allowed`)을 든다(`app/retrieve.py` `_JOINS` · `tests/test_retrieve.py` 가 본다). 한 인스턴스라 한 SQL 로 된다.
- **GraphDB 가 풀 문제가 없다** — 조문의 구조는 깊이가 정해진 나무(조 · 항 · 호 · 목)이고, 판정에 필요한 것은 「이 문장의 근거가 어느 좌표인가」다. 경로 탐색 질의가 없다.
- 대가 — 청크가 수만 단위를 넘으면 전용 벡터DB 보다 느릴 수 있다. 지금 규모(6,686)에서는 해당하지 않는다. 이 PC DB 에서 `chunk_embedding` 은 74 MB, `chunk` 는 21 MB 다(색인 포함 · 10-21 조회).

3. 저장 구조



> 도면 E-03 (2026-10-21 판) — 저장 구조(이 절)와 검색 구조(§5)를 한 장에 그렸다. 10-06 판 도면의 수(청크 6,667 · 사전 506)를 이 판에서 현행 값으로 바꿨다.

표	무엇	요점
`chunk`	조문 청크	인용 단위 `text` 와 검색 문맥 `context` 를 갈랐다 · 조 · 항 · 호 · 목 좌표 · 쪼갠 조각 번호(`part_no` · `part_total`) · 법 축 `law` · 적용 제외 목의 부모 경로 `exempt_of` · 좌표를 막는 이유 `coord_block` (0025) · 원천 조각으로의 외래키
`chunk_embedding`	벡터	`embedding vector(1024)` · `model_id` · `input_sha256` — 청크 삭제에 캐스케이드
`chunk.tsv`	어휘 검색용 생성칸	`to_tsvector('simple', context + text)` · GIN 인덱스 `ix_chunk_tsv`

이 PC DB 의 색인(10-21 · `pg_indexes`) — `chunk` : 기본키 · `ix_chunk_current` (법 ID · 조 · 현행만) · `ix_chunk_fragment` · `ix_chunk_tsv` (GIN) / `chunk_embedding` : 기본키뿐. 벡터 근사 색인(HNSW · ivfflat)은 없다.

- **임베딩 입력은 `context + text` 다**(`scripts/embed.py` `embed_input()`). `context` 는 조문의 호이면 조 제목 + 항 본문, 항이면 조 제목이고(09-12 · 0008), 별표는 별표 제목 + 상위 항목이다(09-24) — 글자 몇 개짜리 짧은 목록 청크가 벡터 상위를 차지해서 붙였다(`preprocess/chunk.py` `_context` · `_annex_context`).

- **모델 이름을 코드에 상수로 두지 않는다** — 질의를 인코딩할 때 `chunk_embedding.model_id` 를 DB 에서 읽어 저장된 벡터를 만든 그 모델을 쓴다. 질의 벡터와 저장 벡터가 다른 모델이면 거리는 뜻이 없는 수가 된다. 벡터 질의도 `e.model_id = %s` 로 같은 모델의 벡터만 비교한다.
- **로드해도 되는 모델은 하나다** — DB 에서 읽은 이름이라도 허용 목록(`ALLOWED_MODELS = {"nlpai-lab/KURE-v1"}`) 밖이면 로드하지 않는다(보안 점검 · D-212). 2026-10-15 부터 질의 임베딩(판정 그래프와 `POST /search` 가 같이 쓰는 `encode`)은 모델을 이 기기의 **로컬 캐시에서만** 올린다(`local_files_only` · 요청 중 외부 내려받기 없음). 모델을 내려받는 것은 적재 도구 `scripts/embed.py` 뿐이다. 캐시가 없으면 벡터 갈래가 멈추고 어휘 갈래만 돌며, 그 사실을 검색 상태에 싣는다(D-344 ⑦).
- **입력 해시를 벡터마다 적는다** — 재임베딩을 중간에 멈추면 두 벡터 공간이 한 표에 섞인다. 검색 코드가 「일부만 비어 있음」을 섞임으로 보고 막는다(`check_inputs`).
- **차원이 한 곳에만 있다** — `app/settings.py` `embed_dim` 1024 와 `db/schema.sql` 의 `vector(1024)` 가 같아야 한다.
- **법 축** — 청크의 법은 법 ID 로 정한다(이름의 낱말로 정하지 않는다). 값은 넷이다(D-271).
- **적용 제외 목**(`exempt_of` · 0021 · D-238 개정 (나)) — 「다만 … 제외한다」의 하위 목은 해당하면 위반이 아닌 경우다. 검색은 그 청크를 그대로 찾고, 위반 근거 좌표는 부모 목으로 올린다(`app/retrieve.py` `basis_citation`). 이 PC DB 에서 값이 있는 청크는 18 이다.
- **좌표 자격 칸**(`coord_block` · 0025 · 2026-10-12 · D-293 개정) — 겹치거나 순서가 깨진 좌표는 인용을 세우지 않는다. 목 글자표를 「가 ~ 하」에서 「거 ~ 허」까지 넓혔다(`app/retrieve.py` `_JO` · 짝 `preprocess/law_norm.py`). 이 PC DB 에서 막힌 청크는 40(겹침 24 · 순서 16)으로 원장 10-13(B) ① 의 40 과 같다.

4. 적재 결과

4-1. 청크 (이 PC DB · 2026-10-21 조회 · 정본 수는 원장 10-13(B) ①)

	수
전체 `chunk` · `chunk_embedding`	**6,686 · 6,686** (모델 `nlpai-lab/KURE-v1` 하나 · 차원 1024)
법별 — 화장품법 · 식품표시광고법 · 표시광고법 · 건강기능식품법	3,483 · 2,091 · 589 · 523
문서 종류 — 별표 · 법령	3,743 · 2,943
쪼갠 조각(`part_total > 1`)	141
임베딩 입력 토큰 512 초과	3 (최대 564)
파생물 파일 `data/derived/chunks.jsonl`	6,686 줄 · sha256 앞 12 자 `e99bd36afc3e` (파생물 원장 기록 10-18)

- **6,667 에서 6,686 으로 바뀐 까닭** — 2026-10-13 정본 기기(클론 B)에서 D-293 개정(좌표 자격 칸 · 목 글자표 「거 ~ 허」 · 형제 순서 검사)을 집행하고 청크를 다시 만들었다. 조문 노드 3,798 · 별표 노드 3,732 에서 청크가 6,667 → **6,686**, 쪼갠 조각이 143 → 141 이 됐고, `embed` 가 보낸 행 = DB = 임베딩 6,686 으로 「DB 가 선언과 같다」를 확인했다. 청크 ID 가 바뀐 옛 청크 258 행은 거뒀다(원장 10-13(B) ①). 같은 날 회사 기기(클론 A)에서도 같은 6,686 이 나왔다(원장 10-13(A) ①).
- 이 PC DB 는 정본이 아니라 복제본이지만, 위 수(6,686 · 막힌 좌표 40 · 적용 제외 18 · 쪼갠 141 · 512 초과 3)가 원장 10-13(B) ① 의 정본 기기 수와 모두 같다. 문서 종류별 수(별표 3,743 · 법령 2,943)는 원장 10-13(A) ⑦(회사 기기 로컬 DB)과 같다. 법별 수는 원장에 따로 적힌 곳이 없어 이 PC DB 조회값이다.
- **색인에는 법령 · 행정규칙 · 별표만 있다** — 6,686 청크가 전부 법령 원천(`law_go_kr`)이다(이 PC DB). 사례(결정문 · 사례집 · 해설서)는 한 건도 색인돼 있지 않다. 사례는 학습 · 평가 라벨로만 쓴다.
- 건강기능식품법 청크는 참고 전용이다(D-271 ⑥). 판정 그래프에는 이 법의 노드가 없어(`app/graph.py` `LAW_OF_NODE` · 법별 노드가 막는다) 위반 근거로도 화면 참고 조문으로도 고르지 않는다. `POST /search` 결과에는 나온다.
- **인용 좌표가 서는 청크 3,430 / 6,686(51%)** — 조문 1,983 · 별표 505 · 공정위 산문 고시 942(원장 10-13(B) ①). 2026-10-21 이 PC DB 에서 검색 코드의 좌표 함수(`retrieve.citation`)로 다시 세어도 3,430(법령 2,925 · 별표 505)이다. 10-06 판의 2,478(37%)에서 늘었다. 나머지는 검색에는 나오지만 화면 참고 조문으로 싣지 않는다(D-224).
- **임베딩 입력 토큰** — 최대 564 · 512 초과 3 행이다. 상한(512)은 리랭커 모델이 정하는데 리랭커를 아직 고르지 않았으므로, 입력 토큰에 상한을 걸지 않고 적재할 때마다 재기만 한다(D-200).
- **시행일 칸은 비어 있다** — 현행 청크 전부가 `effective_date IS NULL` 이다(원장 10-12 ① · 이 PC DB 6,686). 검색도 시행일을 보지 않는다. 「모름」이지 「전부 시행 중」이 아니다.

4-2. 함께 적재되는 관계형 표 (판정이 벡터와 같이 읽는 것)

표	행	기준
`dict_entry` 사전	**4,342**(단독판정 4,189)	사전 판 `2ff0168c4a28` · 원장 10-19 ① · 이 PC DB 같은 수

표	행	기준
`golden_sample` 골든셋	8,383	골든 판 `94a79fe34363` · 학습 5,894 · 평가 분할 2,489 · 원장 10-18 ⑭ · 이 PC DB 같은 수. 평가 분할은 10-14 인코더 단독으로 한 번 쟀다(보고 전용). 판정 그래프의 봉인 시험지는 10-15 재분할(D-347) 뒤 그중 1,141 행이고 열지 않았다
`sanction_rule` 제재 규칙	38	서명됨 · 원장 10-06 클론 A ① · 이 PC DB 38
`violation_article` 위반 조문	14	이 PC DB
`product_fact` 제품 사실	447	이 PC DB
`penalty_rule` · `penal_clause`	0	표는 있으나 적재기가 없다(이 PC DB)

- 적재는 한 트랜잭션이다 — 한 곳에서 멈추면 전부 되돌린다.
- 2 인 확인이 끝나지 않은 원천은 `source` 표에 들어가지 못하고 그 파생물도 적재되지 않는다 — DB 제약(`ck_source_four_eyes`)이 막는다. 못 들어간 것은 넣지 않고 원천별로 센다(D-220). 레지스트리 등재 70 중 적재된 원천은 53 이다(원장 10-06 클론 A · 이 PC DB 53).
- 위험도는 벡터에 넣지 않았다 — 처분 기준표를 평문으로 벡터에 넣으면 표 구조가 깨진다. 관계형 표(`sanction_rule`)로 든다.

5. 검색 구조

```
질의 문장 ─┬── 벡터 갈래   KURE-v1 임베딩 → 코사인 거리 순(pgvector <=>)
           └── 어휘 갈래   조사를 깎은 검색어(OR · 접두어) → tsv 일치 · ts_rank_cd 순
                  │
  (POST /search)  └── 두 갈래 RRF → 규범당 상한(2) → 상위 5
  (판정 그래프)   └── retrieve: 법 필터 없이 한 번 · 법마다 후보 50 · 두 갈래 따로 보관
                      ├── 법별 노드: 자기 법만 거른 뒤 RRF → 규범당 상한(2) → 참고 범위 → 좌표 서는 것 상위 5
                      └── references(판정 뒤): 유형이 정해진 뒤 (유형 · 법) 문서 안 후보만 같은 규칙으로 다시 고름
```

설정	값	출처
후보 폭 `pool`	50	`[설계]` 기획서 5-6 · 바꾸는 것은 결정이다(D-205)
상위 `top_k`	5	`[설계]` 기획서 5-6 · 법별 노드에서 법마다 쓰는 개수(`LAW_TOP_K`)는 `[임의]` · 받을 개수는 판정 대기(D-291)
융합 상수 `rrf_k`	60	`[문헌]` Cormack et al., SIGIR 2009
규범당 상한 `per_law_cap`	2	`[임의]` · 09-27 · 한 고시가 「면역력」 질의 상위 10 을 다 차지해 금지 조항이 상위 5 밖으로 밀려난 것을 보고 넣었다(`app/settings.py` 주석). 바꾸는 조건 — 질의 30 건 탐침에서 상한이 있는 순위와 없는 순위를 나란히 잰다

(값은 모두 `app/settings.py` `Params` 에 있고 코드 다른 곳에 다시 적지 않는다.)

- **순위는 섞고 점수는 안 섞는다**(D-193) — 벡터 거리와 글자 일치 점수를 더하려면 근거 없는 가중치가 필요하다. RRF 는 순위만 쓰므로(`Σ 1/(k + 순위)`) 가중치가 없다. 한 갈래에만 있는 청크도 그쪽 항만으로 들어온다. 뒤집은 근거는 측정이다 — 벡터 단독으로는 정답 조문이 6 위 · 19 위 · 50 위 밖이었다(09-12).
- **넓게 한 번 찾고 법별로 거른다**(D-291) — 검색은 법별 노드로 갈라지기 전에 문장마다 한 번 돈다(`rt.wide`). 법마다 폭만큼 받고, 갈래마다 **먼저 거른 뒤** 섞는다(섞은 뒤 거르면 순위가 부풀었다). 법 필터로 따로 찾은 것과 후보가 같다 — 124 / 124(원장 09-25 ㊴ · 잰 날 09-28 · 질의 31 × 법 범위 4 · 클론 A 도 같음 ㊵ · 설계와 법별 할당은 ㊲ · ㊳).
- **규범당 상한**(`app/retrieve.py` `diversify`) — 한 규범(`law_id` · 법령이나 고시 하나)이 앞자리를 `per_law_cap` 개까지만 차지한다. 넘친 것은 버리지 않고 뒤로 민다. 단위가 법 축이 아니라 규범이다 — 법 축이면 한 법의 금지 조항과 예외 조항이 한 자리를 다툰다.
- **참고 범위 거름**(2026-10-12 · D-328 · D-329) — 별표는 위임 근거에 「광고」가 있는 것만, 본문은 광고 규범 조문 표와 고시 이름 규칙에 드는 것만 참고 조문으로 싣는다(`app/graph.py` `in_reference_scope`).
- **참고 조문은 판정 뒤에 고른다**(2026-10-19 · D-348 보충 30 · 커밋 `cfa398d`) — 코어 끝 노드 `references` 가 문장 유형(확정 · 인코더 근거 · 후보)의 (유형 · 법) 문서표(`statute.REFERENCE_DOCS`) 안 후보만 남겨 법별 노드와 같은 규칙으로 고른다. 유형이 없는 문장에는 참고 조문을 싣지 않는다. 판정 칸과 근거 조문(호)은 바꾸지 않는다. 정본 기기 측정에서 확장 dev 조건부 743 행의 판정 결과 칸은 모두 그대로였고, 유형이 없는 분기 문장의 참고 조문이 0 이 됐다(원장 10-19(B) ⑬).
- **판정은 검색을 읽지 않는다** — 판정 근거(호)는 사전 인용 · 참조 조항 표(`statute.references` · D-327)에서 오고, 검색 결과는 참고 조문으로만 쓴다(원장 10-12 ④ 코드 대조 · `app/graph.py` `retrieve` 주석 · 보충 30 맥락). 그래서 `[임의]` 값 둘(법마다 받는 근거 수 5 · 규범당 상한 2)은 확정 여부를 바꾸지 않고, 붙는 참고 조문만 바꾼다.
- **벡터가 안 돼도 어휘 결과는 낸다** — 모델을 못 올리면 벡터 갈래를 멈추고 어휘 갈래만 돌린 뒤, 왜 안 됐는지를 검색 상태(`SearchState`)에 함께 낸다. 「의미 검색을 했는데 0 건」과 「의미 검색을 못 했다」를 구별한다.

- `by_article` · `by_annex`(좌표로 꺼냄)는 순위 검색이 아니라 사전 근거 · 참조 조항에 원문을 붙이는 조회다(D-224 ④ · D-327). `by_literal`(「제5호 아목」 같은 기호를 글자 그대로 찾기)은 순위 검색과 섞지 않는 별도 함수로 있으나, 2026-10-21 기준 앱에서 부르는 경로는 없다(D-167).

6. 측정 결과

6-1. 검색 품질 — 탐침

(가) 지어낸 문구 31 건 탐침 (원장 10-12 ① · 클론 B · 청크 6,667 · 후보 폭 50 · 법별 상위 5 · 리랭커 없음 · 게이트 아님)

지표	값
정답이 상위 5 안	7 / 31
정답이 후보 50 안	12 / 31
못 찾은 질의	19

- 09-28 측정과 같은 수가 다시 나왔다. 셈 단위가 질의이고 표본이 30 건 기준(D-40)의 평가가 아니므로 탐침으로만 읽는다.
- 못 찾은 19 건의 정답에는 전부 금지 조항(식품표시광고법 제8조제1항 · 화장품법 제13조제1항 · 표시광고법 제3조제1항의 호)이 들어 있다.

(나) dev 문구 탐침 — 골든셋 근거 호를 정답으로 (원장 10-12 ① · 클론 B · dev 640 행 · 근거 좌표 717 · 청크 6,667 · 게이트 아님)

법	근거 좌표	후보 50 안	화면 상위 5
표시광고법	435	2.8%	1.1%
식품표시광고법	215	1.4%	0.5%
화장품법	67	3.0%	0
합	717	17 (2.4%)	6 (0.8%)

- 10-13 청크 재생성(6,686) 뒤 같은 도구로 다시 쟀다 — 후보 안은 그대로(2.8% · 1.4% · 3.0%), 화면 상위 5 는 표시광고 0.0% · 식품 0.5% 였다(원장 10-13(B) ①).
- 읽기 — 검색은 문구와 낱말이 겹치는 청크(성분 · 기능성 · 표시 기준)를 올리고, 추상적으로 적힌 금지 조항은 후보 50 에도 거의 들지 못한다. 후보에 없는 조항은 순서를 바꿔도 올라오지 않으므로 리랭커가 고칠 수 있는 자리가 아니다. 이 결과가 판정 근거를 검색에서 떼어 (유형 × 법) 표에서 가져오는 근거가 됐다(D-348 보충 30 의 전제).
- 정답이 금지 조항(호)뿐이라 고시 · 지침 같은 참고 조문의 쓸모는 이 수로 알 수 없다. 10-13 이후 검색 품질은 다시 재지 않았다(원장에 그 뒤 측정 기록이 없다).

6-2. 응답시간

(가) 판정 요청 1번 — 현재 채택 인코더 포함 (원장 10-20 ① · 정본 기기(클론 B) Windows · `scripts/latency_probe.py` · 문구 1 건 · 품목 모름 · 예열 뒤 3 번 중앙값 · 인코더 D4-20 · 게이트 아님)

구간	시간
DB 연결	15 ms
그래프 실행(invoke)	522 ms

구간	시간
요청 전체	**542 ms**
└ 노드 retrieve(검색)	291 ms
└ 노드 encode(인코더)	115 ms
└ 노드 match_dict(사전)	104 ms
└ 노드 references(참고 조문)	15 ms
예열(첫 요청 · 모델 로드)	26,744 ms

- DB 호스트를 `localhost` 로 두면 IPv6 를 먼저 시도해 연결에 3,130 ms 가 들었다(요청 전체 3,674 ms). `127.0.0.1` 로 바꾼 뒤 15 ms 가 됐고, 기본값(`app/settings.py` `DEFAULT_DATABASE_URL` · `.env.example`)도 함께 고쳤다.
- 검색 노드(291 ms)가 여전히 노드 중 가장 크다. 그래프 실행의 약 56%(291 / 522 · 계산)다.

(나) 검색 한 번의 안쪽 (원장 10-02 ⑨ · 클론 A(회사 기기) · CPU · 문장 100 개 · 예열 제외 · 청크 6,667 · 인코더 없음)

검색 한 번의 안쪽	p50	p95	비중(p50)
임베딩(한 문장)	246.8 ms	378.5 ms	73.4%
벡터 질의	67.8 ms	89.2 ms	20.2%
어휘 갈래	4.7 ms	84.0 ms	1.4%
검색 한 번	336.2 ms	497.4 ms	—

- 검색 시간의 약 3/4 이 **임베딩 계산**이다. 문장 20 개를 한 번에 묶어도 12% 만 준다 — 호출 횟수가 아니라 계산량이다.
- 벡터 질의는 문장당 68 ms 다(인덱스 없음 · 순차 스캔).
- 참고 — 10-12 탐침(31 질의 · 클론 B)의 조회 중앙값은 187.4 ms 였다(원장 10-12 ①).
- 모든 수는 개발 기기의 수다. 배포 서버 · 동시 요청 · 부하 시험은 재지 않았다. 평가 실행의 행당 시간은 다른 측정과 겹쳐 돌았으므로 응답 시간으로 쓰지 않는다.

7. 지키는 장치

장치	무엇을 막나
`tests/test_retrieve.py` (시험 94 · 게이트 표시 69)	거버넌스 조인이 빠진 질의 · 모델 대조 없는 벡터 질의 · 순위 융합이 한쪽 갈래를 버리는 것
`tests/test_embed_upsert.py` (6) · `tests/test_embed_sweep.py` (7)	넣는 칸이 갱신에서 빠지는 것 · 선언에 없는 청크가 남는 것
`tests/test_chunk_ratchet.py` (9)	청크가 조용히 줄어드는 것 — 크게 줄면 `--allow-shrink` 로 사람이 보고 받아들일 때만 적재한다
허용 모델 목록 · 로컬 캐시 전용 로드	DB 문자열로 임의 모델이 로드되는 것 · 판정 중 외부 요청
입력 해시 · 모델 ID	두 벡터 공간이 한 표에 섞이는 것
파생물 원장 `data/derived_manifest.jsonl`	기기마다 청크 판이 갈리는 것 — 정본 기기가 올리고 사본이 받는다

(시험 수는 2026-10-21 기준 커밋 `ksr b54246d` 에서 `pytest --collect-only` 로 센 수다(게이트 표시는 `-m gate`). 이번에 실행하지는 않았다.)

8. 남은 것

1. **리랭커** — 봉인 평가 뒤 안건이다. 자리는 판정이 아니라 (유형 · 법)으로 좁힌 참고 조문 후보 안의 순서다. 0.6B(`bge-reranker-v2-m3`)와 0.3B(`bge-reranker-base`)를 나란히 재고, 「쓸모 있는 참고 조문」 라벨 30 이상을 만든 뒤 고른다(D-348 보충 30 결정 4 · 0.3B 후보는 기획서 7-3 · D-84).
2. **임베딩 비용** — 응답 시간에서 검색 노드가 가장 크다(291 ms · 원장 10-20 ①). 검색을 판정 뒤로 옮겨 유형이 선 문장만 찾는 안은 봉인 뒤 안건이다(보충 30 결정 4).
3. **검색 품질 재측정** — 금지 조항이 후보 50 에 거의 들지 못한다(§ 6-1). 10-13 이후 재측정이 없고, 참고 조문의 쓸모는 판독이 필요하다.
4. **법별 근거 수 k** — `LAW_TOP_K` 5 는 `[임의]` 이고 판정 대기다(D-291).
5. **시행일** — 청크의 시행일 칸이 비어 있어 기획서의 「개정일 필터」가 서 있지 않다(원장 10-12 ①).
6. **임베딩 모델 커밋 고정** — `embed_revision` 이 비어 있어 모델 커밋이 고정되지 않았다(`app/settings.py`).
7. **벡터 인덱스** — 청크가 만 단위가 되면 실측해서 만든다(`app/retrieve.py` 머리말).
8. **배포 서버 DB** — 배포용 RDS PostgreSQL 16 + pgvector 구성은 배포 담당 브랜치(origin/ssm)에 기록이 있고, 이 기준 커밋에는 아직 들어오지 않았다. 배포 서버에서 청크 수 · 응답 시간은 이 문서에서 확인하지 않았다.

10-06 판 「남은 것」 중 정리된 것 — 인용 좌표 확장(2,478 → 3,430 · 10-13 집행), 검색이 판정에 주는 몫(판정은 검색을 읽지 않는다 · 10-12 코드 대조 · 10-19 보충 30), DB 행 수 재조회(이 판에서 이 PC DB 로 조회 · 원장 수와 같음).

> **근거** — 스키마 `db/schema.sql` · `alembic/versions/` (0010 `chunk_tsv` · 0011 `chunk_part` · 0019 `chunk_law` · 0021 `chunk_exempt` · 0025 `chunk_coord_block` · head `0027_judgment_encoder_basis`) · 코드 `app/retrieve.py` · `app/graph.py` (`retrieve` · `_law_node` · `references` · `in_reference_scope`) · `app/settings.py` · `scripts/embed.py` · `preprocess/chunk.py` · 수치 `docs/00_사실원장.md` 09-25 ㊲ ~ ㊴ · 10-02 ⑨ · 10-12 ① ④ · 10-13(B) ① · 10-13(A) ① ⑦ · 10-18 ⑭ · 10-19 ① · 10-19(B) ⑬ · 10-20 ① · 이 PC DB 2026-10-21 읽기 전용 조회 · 판정 `docs/00_설계결정기록.md` D-40 · D-41 · D-84 · D-193 · D-200 · D-205 · D-212 · D-220 · D-224 · D-238 · D-271 · D-291 · D-293 · D-328 · D-329 · D-344 · D-348 보충 30